

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР3: Контейнеризация написанного приложения средствами docker

Срок: 29.04.2026

Задание:

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Ход работы

Система состоит из 7 контейнеров:

Компонент	Назначение	Порт
api-gateway	Единая точка входа для клиента	4000
auth-service	Регистрация, вход, проверка JWT	4001
catalog-service	Каталог ресторанов	4002
booking-service	Бронирование столиков	4003
notification-service	Уведомления о новых бронях	-
postgres	Хранение данных	5432
rabbitmq	Асинхронное взаимодействие сервисов	5672, 15672

Схема взаимодействия:



Принцип разделения:

- Клиент обращается только к API Gateway.
- Каждый бизнес-сервис хранит данные в своей БД (Database per Service).
- Синхронная проверка данных между сервисами идёт через RabbitMQ RPC.
- Асинхронные события (например, «бронь создана») идут через topic exchange.

В Docker адреса сервисов задаются переменными окружения:

- AUTH_SERVICE_URL=http://auth-service:4001
- CATALOG_SERVICE_URL=http://catalog-service:4002

- `BOOKING_SERVICE_URL=http://booking-service:4003`

Локально без Docker используются значения по умолчанию (localhost:4001, 4002, 4003).

3.2. Auth Service (auth-service, порт 4001)

Отвечает за аутентификацию:

- `POST /register` - регистрация пользователя (пароль хэшируется через `bcrypt`);
- `POST /login` - вход, возвращает JWT-токен.

Данные хранятся в PostgreSQL, база **auth_db**.

Дополнительно сервис слушает очередь RabbitMQ **auth.validate.token** и по RPC проверяет JWT. Это нужно booking-service, чтобы не ходить в auth по HTTP напрямую.

3.3. Catalog Service (catalog-service, порт 4002)

Управляет каталогом ресторанов:

- `POST /restaurants` - создание ресторана.

Данные хранятся в базе **catalog_db**.

Также слушает очередь **catalog.restaurant.check**: по RPC возвращает, существует ли ресторан и активен ли он.

3.4. Booking Service (booking-service, порт 4003)

Основной бизнес-сервис:

- `POST /reservations` - создание брони.

При создании брони выполняется цепочка проверок:

1. Проверка JWT через RPC в auth-service.
2. Проверка ресторана через RPC в catalog-service.
3. Сохранение брони в **booking_db**.
4. Публикация события reservation.created в RabbitMQ.

HTTP-эндпоинт у booking-service один, но внутри он координирует работу нескольких сервисов.

3.5. Notification Service

Не имеет HTTP-порта. Это **фоновый consumer**: подписан на exchange reservation.events с routing key reservation.created.

Когда booking-service создаёт бронь, notification-service получает событие и пишет уведомление в лог. В реальной системе здесь могла бы быть отправка email/SMS/push.

3.6. Shared-модуль

Общий код в папке shared/:

- rabbitmq.ts - подключение к RabbitMQ, RPC (rpcCall, setupRpcServer), pub/sub (publishEvent, subscribeEvents);
- types.ts - общие TypeScript-типы для сообщений между сервисами.

Этот модуль подключается в сервисы через относительные импорты и копируется в Docker-образ вместе с сервисом.

Взаимодействие через RabbitMQ

В системе используются два паттерна.

4.1. RPC (синхронный запрос-ответ)

Используется, когда booking-service должен **сразу получить ответ** от другого сервиса.

Пример - создание брони:

1. booking-service отправляет сообщение в очередь auth.validate.token.
2. auth-service проверяет JWT и отправляет ответ в temporary reply queue.
3. booking-service получает { isValid, userId, role }.

Аналогично для проверки ресторана через очередь catalog.restaurant.check.

Это не **REST между сервисами**, а обмен сообщениями через брокер с correlation ID.

4.2. Event-driven (асинхронные события)

После успешного сохранения брони booking-service публикует событие:

- Exchange: reservation.events (тип topic)
- Routing key: reservation.created
- Payload: { reservationId, userId, restaurantId, date }

notification-service подписан на это событие и обрабатывает его независимо, без блокировки HTTP-запроса клиента.

Вывод

1. **Контейнеризировано 5 сервисов** - для каждого написан отдельный Dockerfile.
2. **Собран единый docker-compose.yml**, который поднимает приложение вместе с PostgreSQL и RabbitMQ.
3. **Настроена Docker-сеть restorator-network**, благодаря которой сервисы обращаются друг к другу по DNS-именам контейнеров.
4. **Обеспечена изоляция данных** - у каждого микросервиса своя база PostgreSQL.
5. **Реализовано два типа межсервисного взаимодействия:**
 - HTTP через API Gateway для внешних клиентов;
 - RabbitMQ RPC и события для внутренней логики.

Контейнеризация упростила развёртывание: вместо ручного запуска нескольких Node.js-процессов, PostgreSQL и RabbitMQ достаточно одной команды `docker compose up`. Это соответствует идеям микросервисной архитектуры: независимые сервисы, отдельные контейнеры, централизованная оркестрация через Docker Compose.